

Árboles de búsqueda equilibrados. Árboles B

Objetivos

Con el estudio de este capítulo usted podrá:

- Conocer la eficiencia de un árbol de búsqueda.
- Construir un árbol binario equilibrado conociendo el número de claves.
- Construir un árbol binario de búsqueda equilibrado.
- Describir los diversos tipos de movimientos que se hacen cuando se desequilibra un árbol.
- Diseñar y declarar la clase `ArbolEquilibrado`.
- Conocer las características de los árboles B.
- Utilizar la estructura de árbol B para organizar búsquedas eficientes en bases de datos.
- Implementar la operación de búsqueda de una clave en un árbol B.
- Conocer la estrategia que sigue el proceso de inserción de una clave en un árbol B.
- Implementar las operaciones del TAD árbol B en C++.

Contenido

- | | |
|--|---|
| <p>17.1. Eficiencia de la búsqueda en un árbol ordenado.</p> <p>17.2. Árbol binario equilibrado, árbol AVL.</p> <p>17.3. Inserción en árboles de búsqueda equilibrados. Rotaciones.</p> <p>17.4. Implementación de la operación <i>inserción con balanceo y rotaciones</i>.</p> <p>17.5. Definición de un árbol B.</p> <p>17.6. <i>TAD árbol B</i> y representación.</p> <p>17.7. Formación de un árbol B.</p> | <p>17.8. Búsqueda de una clave en un árbol B.</p> <p>17.9. Inserción en un árbol B.</p> <p>17.10. Listado de las claves de un árbol B.</p> <p>RESUMEN.</p> <p>BIBLIOGRAFÍA RECOMENDADA.</p> <p>EJERCICIOS.</p> <p>PROBLEMAS.</p> <p>ANEXOS A y B en página web del libro (lectura recomendada para profundizar en el tema).</p> |
|--|---|

Conceptos clave

- | | |
|--|---|
| <ul style="list-style-type: none">• Altura de un árbol.• Árbol de búsqueda.• Árboles B.• Camino de búsqueda.• Complejidad logarítmica. | <ul style="list-style-type: none">• Equilibrio.• Factor de equilibrio.• Hoja.• Rotaciones. |
|--|---|

INTRODUCCIÓN

En el Capítulo 16 se introdujo el concepto de árbol binario. Se utiliza un árbol binario de búsqueda para almacenar datos organizados jerárquicamente. Sin embargo, en muchas ocasiones, las inserciones y eliminaciones de elementos en el árbol no ocurren en un orden predecible; es decir, los datos no están organizados jerárquicamente.

En este capítulo se estudian tipos de árboles adicionales: los árboles equilibrados o árboles AVL, como también se les conoce, y los árboles B.

El concepto de árbol equilibrado así como los algoritmos de manipulación son el motivo central de este capítulo. Los métodos que describen este tipo de árboles fueron descritos en 1962 por los matemáticos rusos G. M. Adelson - Velskii y E. M. Landis.

Los árboles B se utilizan para la creación de bases de datos. Así, una forma de implementar los índices de una base de datos relacional es a través de un árbol B.

Otra aplicación dada a los árboles B es la gestión del sistema de archivos de los sistemas operativos, con el fin de aumentar la eficacia en la búsqueda de archivos por los subdirectorios.

También se conocen aplicaciones de los árboles B en sistemas de comprensión de datos. Bastantes algoritmos de comprensión utilizan árboles B para la búsqueda por claves de datos comprimidos.

17.1. EFICIENCIA DE LA BÚSQUEDA EN UN ÁRBOL ORDENADO

La eficiencia de una búsqueda en un árbol binario ordenado varía entre $O(n)$ y $O(\log(n))$, dependiendo de la estructura que presente el árbol.

Si los elementos son añadidos en el árbol mediante el algoritmo de inserción expuesto en el capítulo anterior, la estructura resultante del árbol dependerá del orden en que sean añadidos. Así, si todos los elementos se insertan en orden creciente o decreciente, el árbol va a tener todas la ramas izquierda o derecha, respectivamente, vacías. Entonces, la búsqueda en dicho árbol será totalmente secuencial.

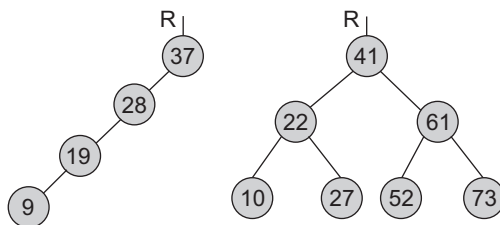


Figura 17.1. Árbol degenerado y equilibrado de búsqueda.

Sin embargo, si la mitad de los elementos insertados después de otro con clave K tienen claves menores de K y la otra mitad claves mayores de K , se obtiene un árbol equilibrado (también llamado balanceado), en el cual las comparaciones para obtener un elemento son como máximo $\log_2(n)$, para un árbol de n nodos.

En los árboles de búsqueda el número promedio de comparaciones que debe de realizarse para las operaciones de inserción, eliminación y búsqueda varía entre $\log_2(n)$, para el mejor de los casos, y n para el peor de los casos. Para optimizar los tiempos de búsqueda en los árboles ordenados surgen los árboles casi equilibrados, en los que la complejidad de la búsqueda es logarítmica, $O(\log(n))$.

17.2. ÁRBOL BINARIO EQUILIBRADO, ÁRBOLES AVL

Un árbol totalmente equilibrado se caracteriza por que la altura de la rama izquierda es igual que la altura de la rama derecha para cada uno de los nodos del árbol. Es un árbol ideal, no siempre se puede conseguir que el árbol esté totalmente balanceado.

La estructura de datos de árbol equilibrado que se utiliza es la árbol **AVL**. El nombre es en honor de Adelson-Velskii-Landis que fueron los primeros científicos en estudiar las propiedades de esta estructura de datos. Son árboles ordenados o de búsqueda que, además, cumplen la condición de balanceo para cada uno de los nodos.

Definición

Un árbol equilibrado o *árbol AVL* es un árbol binario de búsqueda en el que las alturas de los subárboles izquierdo y derecho de cualquier nodo difieren como máximo en 1.

La Figura 17.2 muestra dos árboles de búsqueda, el de la izquierda está equilibrado. El de la derecha es el resultado de insertar la clave 2 en el anterior, según el algoritmo de inserción en árboles de búsqueda. La inserción provoca que se *viola* la condición de equilibrio en el nodo raíz del árbol.

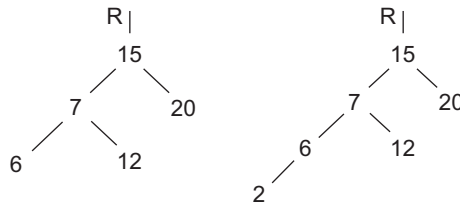


Figura 17.2. Dos árboles de búsqueda, el de la izquierda equilibrado, el otro no.

La condición de equilibrio de cada nodo implica una restricción en las alturas de los subárboles de un árbol AVL. Si v_i es la raíz de cualquier subárbol de un árbol equilibrado, y h la altura de la rama izquierda entonces la altura de la rama derecha puede tomar los valores: $h-1$, h , $h+1$. Esto aconseja asociar a cada nodo el parámetro denominado *factor de equilibrio* o *balance de un nodo*. Se define como la altura del subárbol derecho menos la altura del subárbol izquierdo correspondiente. El factor de equilibrio de cada nodo en un árbol equilibrado puede tomar los valores: 1, -1 o 0. La Figura 17.3 muestra un árbol balanceado con el factor de equilibrio de cada nodo.

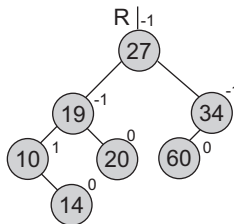


Figura 17.3. Árbol equilibrado con el factor de equilibrio de cada nodo.

A tener en cuenta

La altura o profundidad de un árbol binario es el nivel máximo de sus hojas más uno. La altura de un árbol nulo se considera cero.

17.2.1. Altura de un árbol equilibrado, árbol AVL

No resulta fácil determinar la altura promedio de un árbol AVL, por lo que se determina la altura en el *peor de los casos*, es decir, la altura máxima que puede tener un árbol equilibrado con un número de nodos n . La altura es un parámetro importante ya que coincide con el número de iteraciones que se realizan para *bajar* desde el nodo raíz al nivel más profundo de las hojas. La eficiencia de los algoritmos de búsqueda, inserción y borrado depende de la altura del árbol AVL.

Para calcular la altura máxima de un árbol AVL de n nodos se parte del siguiente razonamiento: ¿cuál es el número mínimo de nodos que puede tener un árbol binario para que se considere reequilibrado con una altura h ? Si ese árbol es A_h , tendrá dos subárboles izquierdo y derecho respectivamente, A_i y A_d cuya altura difiera en 1, supongamos que tienen de altura $h-1$ y $h-2$ respectivamente. Al considerar que A_h es el árbol de menor número de nodos de altura h , entonces A_i y A_d también son árboles AVL de menor número de nodos, pero de altura $h-1$ y $h-2$ y son designados como A_{h-1} y A_{h-2} . Este razonamiento se sigue extendiendo a cada subárbol y se obtienen árboles equilibrados como los de la Figura 17.4.

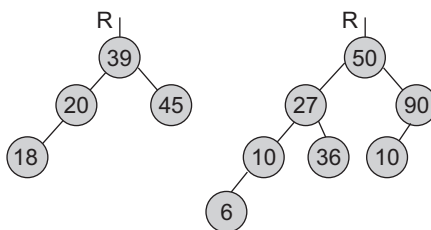


Figura 17.4. Árboles de Fibonacci.

La construcción de árboles binarios equilibrados siguiendo esta estrategia puede representarse matemáticamente:

$$A_h = A_{h-1} + A_{h-2}$$

La expresión matemática expuesta tiene una gran similitud con la ley de recurrencia que permite encontrar números de Fibonacci, $a_n = a_{n-1} + a_{n-2}$. Por esa razón, a los árboles equilibrados contruidos con esta ley de formación se les conoce como *árboles de Fibonacci*.

El objetivo que se persigue es encontrar el número de nodos, n , que hace la altura máxima. El número de nodos de un árbol es la suma de los nodos de su rama izquierda, rama derecha más uno (la raíz). Si ese número es N_h se puede escribir:

$$N_h = N_{h-1} + N_{h-2} + 1$$

donde $N_0 = 1$, $N_1 = 2$, $N_2 = 4$, $N_3 = 7$, y así sucesivamente. Se observa que los números $N_h + 1$ cumplen la definición de los números de Fibonacci. El estudio matemático de la función generadora de los números de Fibonacci permite encontrar esta relación:

$$N_h + 1 \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^h$$

Tomando logaritmos se encuentra la altura h en función del número de nodos, N_h :

$$h \approx 1.44 \log(N_h)$$

Como conclusión, el árbol equilibrado de n nodos menos denso tiene como altura $1.44 \log n$. Como n es el número de nodos en el *peor de los casos* de árbol AVL de altura h , se puede afirmar que la complejidad de una búsqueda es $O(\log n)$.

A tener en cuenta

La altura de un árbol binario perfectamente equilibrado de n nodos es $\log n$. Las operaciones que se aplican a los árboles AVL no requieren más del 44 por cien de tiempo (en el caso más desfavorable) que si se aplican a un árbol perfectamente equilibrado.

EJERCICIO 17.1. Se tienen n claves que se van a organizar jerárquicamente formando un árbol equilibrado. Escribir un programa para formar el árbol AVL siguiendo la estrategia de los árboles de Fibonacci descrita en la sección anterior.

La formación de un árbol balanceado de n nodos se parece mucho a la secuencia de los números de Fibonacci:

$$a(n) = a(n-2) + a(n-1)$$

Un *árbol de Fibonacci* (árbol equilibrado) puede definirse:

1. Un árbol vacío es el árbol de Fibonacci de altura 0.
2. Un nodo único es un árbol de Fibonacci de altura 1.
3. Si A_{h-1} y A_{h-2} son árboles de Fibonacci de alturas $h-1$ y $h-2$, entonces

$$A_h = \langle A_{h-1}, x, A_{h-2} \rangle \text{ es árbol de Fibonacci de altura } h.$$

El número de nodos, A_h , viene dado por la sencilla relación recurrente:

$$N_0 = 0$$

$$N_1 = 1$$

$$N_h = N_{h-1} + 1 + N_{h-2}$$

Para conseguir un árbol AVL con un número dado, n , de nodos de mínima altura hay que distribuir equitativamente los nodos a la izquierda y a la derecha de un nodo dado. En definitiva, es seguir la relación de recurrencia anterior, que expresada recursivamente:

1. Crear nodo raíz.
2. Generar el subárbol izquierdo con $n_1 = n/2$ nodos del nodo raíz utilizando la misma estrategia.

3. Generar el subárbol derecho con $n_d = n - n_i - 1$ nodos del nodo raíz utilizando la misma estrategia.

En este ejercicio, el árbol no va a ser de búsqueda. Simplemente, un árbol binario de números enteros que son leídos del teclado pero que es de mínima altura. El árbol será de búsqueda, si los números que se leen están ordenados crecientemente.

El método público `ArbolFibonacci()` de la clase `ArbolBinario` realiza una llamada al método privado `arbolFibonacci()` de la clase `ArbolBinario` que retorna un puntero a la clase `Nodo`.

```
void ArbolBinario::ArbolFibonacci(int n)
{
    raiz = arbolFibonacci(n);
}

Nodo* ArbolBinario::arbolFibonacci(int n)
{
    int nodosIz, nodosDr;
    int clave;
    Nodo *nuevoRaiz;

    if (n == 0)
        return NULL;
    else
    {
        nodosIz = n / 2;
        nodosDr = n - nodosIz - 1;
        // nodo raíz con árbol izquierdo y derecho de Fibonacci
        cin >> clave;
        nuevoRaiz = new Nodo(arbolFibonacci(nodosIz), clave,
                             arbolFibonacci(nodosDr));
    }
    return nuevoRaiz;
}
```

El método público `dibujarArbol()` de la clase `ArbolBinario`, se encarga de representar en el dispositivo estándar de salida la información almacenada en cada uno de los nodos del árbol. Usa un parámetro `h` para formatear la salida.

```
void ArbolBinario::dibujarArbol(Nodo *r, int h)
{
    /*
     * escribe las claves del arbol de fibonacci; h establece
     * una separación entre nodos
     */
    int i;
    if (r != NULL)
    {
        dibujarArbol(r->subarbolIzdo(), h + 1);
        for (i = 1; i <= h; i++)
            cout << " ";
        cout << r->valorNodo() << endl;
        dibujarArbol(r->subarbolDcho(), h + 1);
    }
}
```

El programa principal realiza las llamadas correspondientes.

```
int main()
{
    ArbolBinario arbolFib;
    int n;
    do {
        cout << "Número de nodos del árbol: ";
        cin >> n;
    } while (n <= 0);
    arbolFib.ArbolFibonacci(n);
    cout << "Árbol de Fibonacci de mínima altura:\n";
    arbolFib.dibujarArbol(arbolFib.Oraiz(), 1);
    return 0;
}
```

17.3. INSERCIÓN EN ÁRBOLES DE BÚSQUEDA EQUILIBRADOS: ROTACIONES

Los árboles equilibrados, árboles AVL, son árboles de búsqueda y, por consiguiente, para añadir un elemento se ha de seguir el mismo proceso que en los árboles de búsqueda. Se compara la nueva clave con la clave del raíz, continúa por la rama izquierda o derecha según sea menor o mayor (describe el *camino de búsqueda*), termina insertándose como nodo hoja. Esta operación, como ha quedado demostrado al determinar la altura en el peor de los casos, tiene una complejidad logarítmica. Sin embargo, la nueva inserción puede hacer que aumente la altura de una rama, de manera que cambie el factor de equilibrio del nodo raíz de dicha rama. Este hecho hace necesario que el algoritmo de inserción, *regrese* por el *camino de búsqueda* actualizando el factor de equilibrio de los nodos. La Figura 17.5 muestra un árbol equilibrado y el mismo árbol justo después de la inserción de una nueva clave que provoca que *rompa* la condición de balanceo.

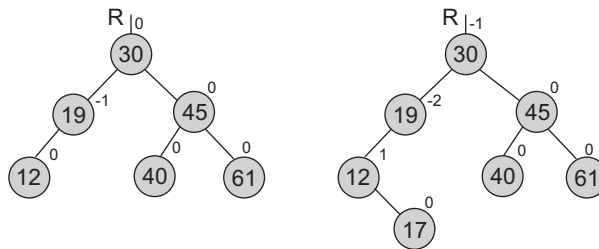


Figura 17.5. Árbol equilibrado; el mismo después de insertar la clave 17.

A recordar

Una inserción de una nueva clave, o un borrado, puede destruir el criterio de equilibrio de varios nodos del árbol. Se debe recuperar la condición de equilibrio del árbol antes de dar por finalizada la operación para que el árbol siga siendo equilibrado.

La estructura del nodo en un árbol equilibrado es una *extensión* de la declarada para un árbol binario. Para determinar si el árbol está equilibrado debe de manejarse información

relativa al balanceo o factor de equilibrio de cada nodo. Por esta razón se añade al nodo un campo más: el factor de equilibrio (*fe*). Este atributo puede tomar los valores: -1, 0, +1.

La clase *NodoAvl* es similar a la clase *Nodo* de los árboles binarios de búsqueda. Solamente se le añade un nuevo atributo entero *fe*, a los que ya disponía la clase *Nodo*. Además, se añaden las funciones miembro encargadas de obtener el valor del nuevo atributo coma la de modificar el valor del nuevo atributo, así como los correspondientes métodos

```
class NodoAvl
{
protected:
    Tipoelemento dato;
    NodoAvl *izdo;
    NodoAvl *dcho;
    int fe;
public:
    NodoAvl(Tipoelemento valor)
    {
        dato = valor;
        izdo = dcho = NULL;
        fe = 0;
    }
    NodoAvl(Tipoelemento valor, int vfe)
    {
        dato = valor;
        izdo = dcho = NULL;
        fe = vfe;
    }
    NodoAvl(NodoAvl* ramaIzdo, Tipoelemento valor, NodoAvl* ramaDcho)
    {
        dato = valor;
        izdo = ramaIzdo;
        dcho = ramaDcho;
        fe = 0;
    }
    NodoAvl(NodoAvl* ramaIzdo, int vfe, Tipoelemento valor,
            NodoAvl* ramaDcho)
    {
        dato = valor;
        izdo = ramaIzdo;
        dcho = ramaDcho;
        fe = vfe;
    }
    // operaciones de acceso
    Tipoelemento valorNodo(){ return dato; }
    NodoAvl* subarbolIzdo(){ return izdo; }
    NodoAvl* subarbolDcho(){ return dcho; }
    void nuevoValor(Tipoelemento d){ dato = d; }
    void ramaIzdo(NodoAvl* n){ izdo = n; }
    void ramaDcho(NodoAvl* n){ dcho = n; }
    void visitar(){ cout << dato << endl;}
    void Pfe(int vfe) { fe = vfe;}
    int Ofc(){ return fe;}
};
```


Las operaciones a realizar con un árbol de búsqueda equilibrado son las mismas que con un árbol de búsqueda. Debido a los cambios del nodo, se declara la clase `ArbolAvl` sin relacionarla con `ArbolBinarioBusqueda`.

```
class ArbolAvl
{
    NodoAvl* raiz;
    ArbolAvl()
    {
        raiz = NULL;
    }
    ArbolAvl(NodoAvl * r)
    {
        raiz = r;
    }
    NodoAvl* Oraiz ()
    {
        return raiz;
    }
    void Praiz( NodoAvl *r)
    {
        raiz = r;
    }
    ...
};
```

17.3.1. Proceso de inserción de un nuevo nodo

Inicialmente, se aplica el algoritmo de inserción en un árbol de búsqueda, éste sigue el *camino de búsqueda* hasta llegar al fondo del árbol y se enlaza como nodo *hoja* y con *factor de equilibrio* 0. Pero el proceso no puede terminar, es necesario recorrer el *camino de búsqueda* en sentido contrario, hacia la raíz, para actualizar el campo adicional *factor de equilibrio*. Después de una inserción sólo los nodos que se encuentran en el camino de búsqueda pueden haber cambiado el factor de equilibrio.

La actualización del *factor de equilibrio* (fe) puede hacer que éste *mejore*. Esto ocurre cuando un nodo está descompensado a la izquierda y se inserta el nuevo nodo en la rama izquierda, al crecer en altura dicha rama el fe se hace 0, se ha *mejorado* el equilibrio. La Figura 17.6 muestra el árbol a) en el que el nodo 90 tiene fe = 1; en el árbol b), después de insertar el nodo con clave 60, el nodo 90 tiene fe = 0.

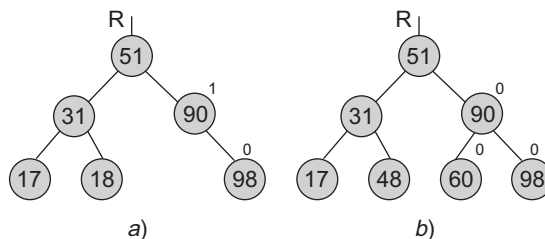


Figura 17.6. Mejora en la condición de equilibrio al insertar un nuevo nodo con clave 60.

La actualización del fe de un nodo del *camino de búsqueda* que, originalmente, tiene las ramas izquierda y derecha de la misma altura ($hRi = hRd$), no va a causar romper el criterio de equilibrio.

Al actualizar el nodo cuyas ramas izquierda y derecha del árbol tienen altura diferente, $|hRi - hRd| = 1$, si se inserta el nodo en la rama más alta rompe el criterio de equilibrio del árbol, la diferencia de altura pasa a ser 2 y es necesario reestructurarlo.

Hay cuatro casos que se deben tener en cuenta al reestructurar un nodo A, según dónde se haya hecho la inserción:

1. Inserción en el subárbol izquierdo de la rama izquierda de A.
2. Inserción en el subárbol derecho de la rama izquierda de A.
3. Inserción en el subárbol derecho de la rama derecha de A.
4. Inserción en el subárbol izquierdo de la rama derecha de A.

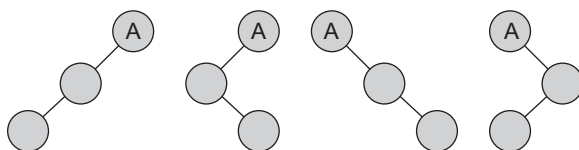


Figura 17.7. Cuatro tipos de reestructuraciones del equilibrio de un nodo.

El primer y tercer caso (*izquierda-izquierda*, *derecha-derecha*) se resuelven con una *rotación simple*. El segundo y cuarto caso (*izquierda-derecha*, *derecha-izquierda*) se resuelven con una *rotación doble*.

La rotación simple implica a dos nodos, el nodo A (nodo con $|fe| = 2$) y el descendiente izquierdo o derecho según el caso. En la rotación doble están implicados tres nodos, el nodo A, nodo descendiente izquierdo y el descendiente derecho de éste; o bien el caso simétrico, nodo A, descendiente derecho y el descendiente izquierdo de éste.

A recordar

Una reestructuración de los nodos implicados en la violación de *criterio de equilibrio*, ya sea una rotación simple o doble, hace que se recupere el equilibrio en todo el árbol, no siendo necesario seguir analizando los nodos del *camino de búsqueda*.

El proceso de *regresar* por el *camino de búsqueda* termina cuando se llega a la raíz del árbol, o cuando se realiza la reestructuración en un nodo del mismo. Una vez realizada una reestructuración no es necesario determinar el *factor de equilibrio* de los restantes nodos, debido a que dicho factor queda como el que tenía antes de la inserción, ya que la reestructuración hace que no aumente la altura.

17.3.2. Rotación simple

La rotación simple resuelve la *violación del equilibrio* de un nodo *izquierda-izquierda*, simétrica a la *derecha-derecha*. El árbol de la Figura 17.8a) tiene el nodo C con factor de equi-

librio -1 ; el árbol de la Figura 17.8b) es el resultado de insertar el nodo A. Resulta que ha crecido la altura de la rama izquierda, es un desequilibrio *izquierda-izquierda* que se resuelve con una rotación simple, rotación II. La Figura 17.9 es el árbol resultante de la rotación, el nodo B se ha convertido en la raíz, el nodo C su rama derecha y el nodo A continúa como rama izquierda. Con estos movimientos el árbol sigue siendo de búsqueda y se equilibra.

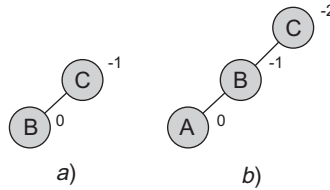


Figura 17.8. Árbol binario AVL y árbol después de insertar nueva clave por la izquierda.

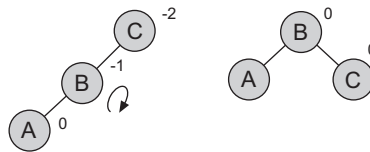


Figura 17.9. Árbol binario después de rotación simple II.

La Figura 17.10 muestra el otro caso de violación de la condición de equilibrio que se resuelve con una rotación simple. Inicialmente, el nodo A tiene como factor de equilibrio $+1$, al insertar el nodo C el factor de equilibrio de A pasa a ser $+2$, se ha insertado por la derecha y, por consiguiente, ha crecido la altura de la rama derecha. La Figura 17.11 muestra la resolución de este desequilibrio, una rotación simple que se puede denominar rotación DD. En el árbol resultante de la rotación, el nodo B se ha convertido en la raíz, el nodo A es su rama izquierda y el nodo C continúa como rama derecha. Con estos movimientos el árbol sigue siendo de búsqueda y queda equilibrado.

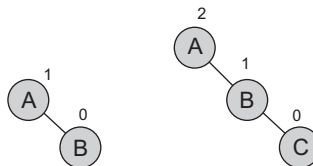


Figura 17.10. Árbol binario AVL y árbol después de insertar nueva clave por la derecha.

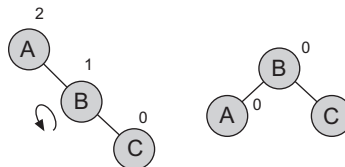


Figura 17.11. Árbol binario después de rotación simple DD.

17.3.3. Movimiento de enlaces en la rotación simple

Los cambios descritos en la rotación simple afectan a dos nodos, el tercero no se modifica, es necesario sólo una rotación. Para la rotación simple a la *izquierda*, rotación II, los ajustes necesarios de los enlaces, suponiendo n la referencia al nodo problema y $n1$ la referencia al nodo de su rama izquierda:

```
n->izdo = n1->dcho;
n1->dcho = n;
n = n1;
```

Una vez realizada la rotación, los factores de equilibrio de los nodos que intervienen siempre es 0, los subárboles izquierdo y derecho tienen la misma altura. Incluso, la altura del subárbol implicado es la misma después de la inserción que antes. La Figura 17.12 muestra estos movimientos de los enlaces.

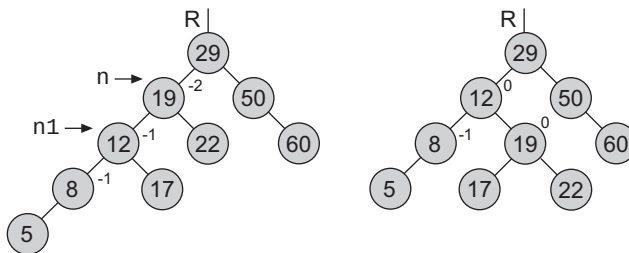


Figura 17.12. Rotación simple a izquierda en un árbol después de insertar la clave 5.

Si la rotación simple es a *derecha*, rotación DD, los cambios en los enlaces del nodo n (con factor de equilibrio +2) y del nodo de su rama derecha, $n1$:

```
n->dcho = n1->izdo;
n1->izdo = n;
n = n1;
```

Realizada la rotación, los factores de equilibrio de los nodos que intervienen es 0. Se puede observar que estos ajustes son los simétricos a los realizados en la rotación II.

17.3.4. Rotación doble

Con la rotación simple no es posible resolver todos los casos de violación del criterio de equilibrio. El árbol de búsqueda de la Figura 17.13a) está desequilibrado, con factores de equilibrio +2, -1 y 0. La Figura 17.13b) aplica la rotación simple, rotación DD. La única solución consiste en *subir* el nodo 40 como raíz del subárbol, como rama izquierda situar al nodo 30 y como rama derecha el nodo 60, la altura del subárbol resultante es la misma que antes de insertar. Se ha realizado una rotación doble, *derecha-izquierda*, en la que intervienen los tres nodos.

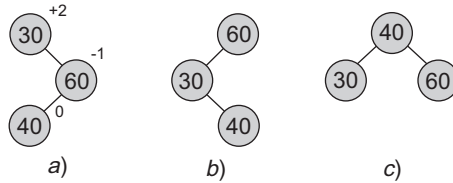


Figura 17.13. a) Árbol después de insertar clave 40. b) Rotación D c) Rotación doble para equilibrar.

La Figura 17.14a) muestra un árbol binario de búsqueda después de insertar la clave 60. Al volver por el *camino de búsqueda* para actualizar los factores de equilibrio, el nodo 75 pasa a tener $fe = -1$ (se ha insertado por su izquierda), el nodo 50 pasa a tener $fe = +1$ y el nodo 80 tendrá como $fe = -2$. Es el caso simétrico al descrito en la Figura 17.13, se reestablece el equilibrio con una rotación doble, simétrica con respecto a la anterior como se muestra en la Figura 17.14b).

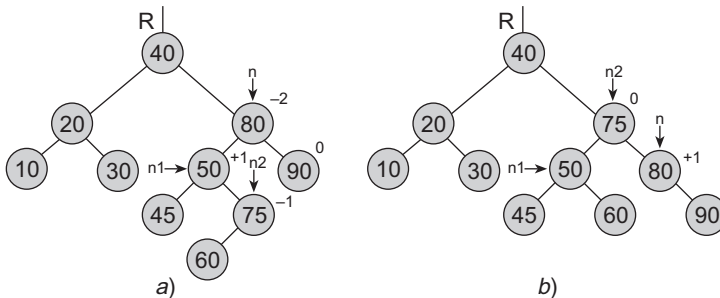


Figura 17.14. a) Árbol después de insertar clave 60. b) Rotación doble, izquierda derecha.

Recuerde

La rotación doble resuelve dos casos simétricos, se pueden denominar rotación ID y rotación DI. En la rotación doble hay que mover los enlaces de tres nodos, el nodo *padre*, el *descendiente* y el *descendiente del descendiente* por la rama contraria.

17.3.5. Movimiento de enlaces en la rotación doble

Los cambios descritos en la rotación doble afectan a tres nodos, el nodo problema n , el descendiente por la rama desequilibrada $n1$, y el descendiente de $n1$ (por la izquierda o la derecha, según el tipo de rotación doble) apuntado por $n2$. En los dos casos simétricos de rotación doble, rotación *izquierda-derecha* (rotación ID) y rotación *derecha-izquierda* (rotación DI), el nodo $n2$ pasa a ser la raíz del nuevo subárbol.

Los movimientos de los enlaces para realizar la rotación ID:

```
n1->dcho = n2->izdo;
n2->izdo = n1;
n->izdo = n2->dcho;
n2->dcho = n;
n = n2;
```

Los factores de equilibrio de los nodos implicados en la rotación ID depende del factor de equilibrio, antes de la inserción, del nodo apuntado por n2, según esta tabla:

Si	$n2 \rightarrow fe = -1$	$n2 \rightarrow fe = 0$	$n2 \rightarrow fe = 1$
	$n \rightarrow fe = 1$	0	0
	$n1 \rightarrow fe = 0$	0	-1
	$n2 \rightarrow fe = 0$	0	0

Los movimientos de los enlaces para realizar la rotación DI (observar la simetría en los movimientos de los punteros):

```
n1->izdo = n2->dcho;  
n2->dcho = n1;  
n->dcho = n2->izdo;  
n2->izdo = n;  
n = n2 ;
```

Los factores de equilibrio de los nodos implicados en la rotación DI también dependen del factor de equilibrio previo del nodo n2, según la tabla:

Si	$n2 \rightarrow fe = -1$	$n2 \rightarrow fe = 0$	$n2 \rightarrow fe = 1$
	$n \rightarrow fe = 0$	0	-1
	$n1 \rightarrow fe = 1$	0	0
	$n2 \rightarrow fe = 0$	0	0

A recordar

La complejidad del algoritmo de inserción de una clave en un árbol de búsqueda AVL es la suma de la complejidad para bajar al nivel de las hojas ($O(\log n)$) más la complejidad en el peor de los casos de la vuelta por el camino de búsqueda, para actualizar el factor de equilibrio de los nodos que es $O(\log n)$, más la complejidad de los movimientos de los enlaces en la rotación, que tiene complejidad constante. En definitiva, la complejidad de la inserción es $O(\log n)$, complejidad logarítmica.

17.4. IMPLEMENTACIÓN DE LA INSERCIÓN CON BALANCEO Y ROTACIONES

La realización de la fase de inserción es igual que la escrita para los árboles de búsqueda. Ahora se añade la fase de actualización de los factores de equilibrio; una vez insertado, se activa un *flag* para indicar que ha crecido en altura, de tal forma que al *regresar* por el *camino de búsqueda* calcule los nuevos factores de equilibrio de los nodos que forman el camino. Cuando la inserción se ha realizado por la rama izquierda del nodo, la altura crece por la izquierda y, por tanto, disminuye en 1 el factor de equilibrio; si se hace por la rama derecha, el factor de equilibrio aumenta en 1. El proceso termina si la altura del subárbol no aumenta. También termina si se produce un desequilibrio, ya que cualquier rotación tiene la propiedad de que la altura del subárbol resultante es la misma que antes de la inserción.

A continuación, se escriben los métodos privados (miembros privados de la clase `ArbolAVL`) que implementan los cuatro tipos de rotaciones y de la operación de inserción. Todos devuelven la referencia al nodo raíz del subárbol implicado en la rotación.

Rotaciones

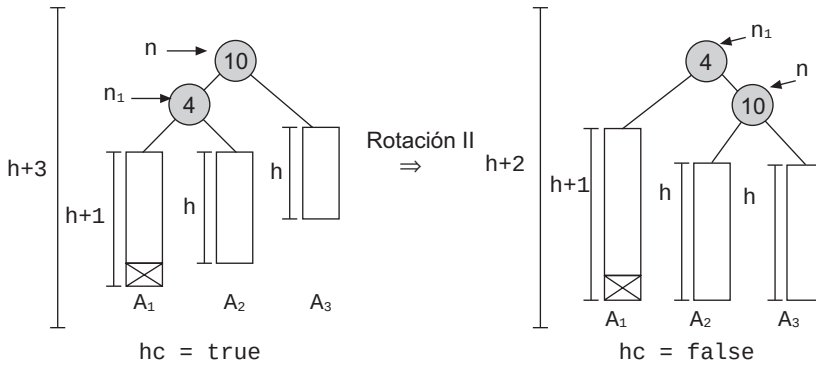


Figura 17.15.¹ Rotación II. Esquema gráfico. Árbol deja de aumentar en altura.

```

NodoAvl* ArbolAvl::rotacionII(NodoAvl* n, NodoAvl* n1)
{
    //Figura 17.5
    n->ramaIzdo(n1->subarbolDcho());
    n1->ramaDcho(n);
    // actualización de los factores de equilibrio
    if (n1->Ofe() == -1) // la condición es true en la inserción
    {
        n->Pfe(0);
        n1->Pfe(0);
    }
}

```

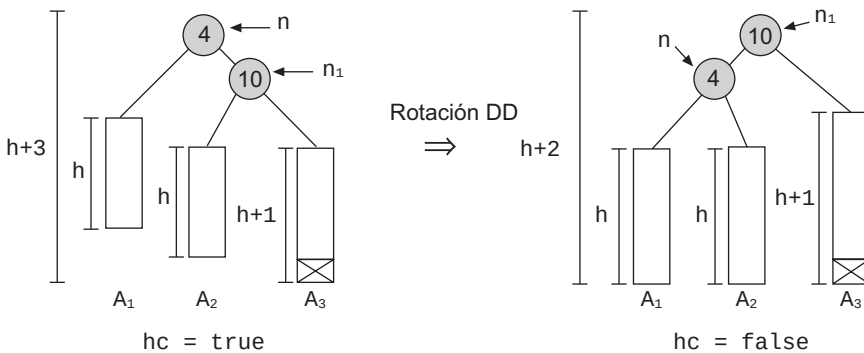


Figura 17.16. Rotación DD. Esquema gráfico. Árbol deja de aumentar en altura.

```

else
{
    n->Pfe(-1);
}

```

^{1,2} En [KRUSE 94] el lector podrá encontrar una excelente referencia para profundizar en temas avanzados de árboles AVL, B y *tries* (árboles de búsqueda lexicográficos. Los códigos de las funciones, procedimientos y programas están escritos) en lenguaje Pascal, pero sus conceptos teóricos y prácticos sirven para la implantación en cualquier lenguaje.

```

    n1->Pfe(1);
}
return n1;
}

NodoAvl* ArbolAvl::rotacionDD(NodoAvl* n, NodoAvl* n1)
{
    //Figura 17.16
    n->ramaDcho(n1->subarbolIzdo());
    n1->ramaIzdo(n);
    // actualización de los factores de equilibrio
    if (n1->Ofe() == +1) // la condición es true en la inserción
    {
        n->Pfe(0);
        n1->Pfe(0);
    }
    else
    {
        n->Pfe(+1);
        n1->Pfe(-1);
    }
    return n1;
}

```

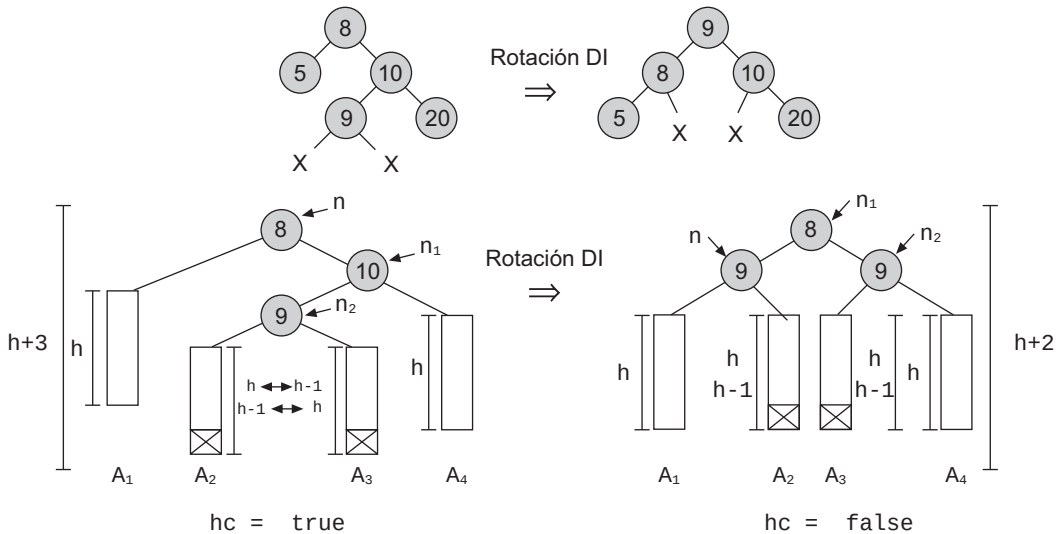


Figura 17.17. Rotación DL. Esquema gráfico. Árbol deja de aumentar en altura^{3,4}.

```

NodoAvl* ArbolAvl::rotacionDI(NodoAvl* n, NodoAvl* n1)
{
    //Figura 17.17
    NodoAvl* n2;

    n2 = n1->subarbolIzdo();
    n->ramaDcho(n2->subarbolIzdo());
}

```

³ Si A_2 tiene altura h , A_3 tiene altura $h-1$; si A_2 tiene altura $h-1$, A_3 tiene altura h . No puede darse en las inserciones que A_2 tenga altura h y A_3 tenga altura h , ya que en este caso no hay desequilibrio en el nodo n_2 .

⁴ Ibid, [KRUSE 84].


```

n2->ramaIzdo(n);
n1->ramaIzdo(n2->subarbolDcho());
n2->ramaDcho(n1);
    // actualización de los factores de equilibrio
if (n2->Ofe() == +1)
    n->Pfe(-1);
else
    n->Pfe(0);

if (n2->Ofe() == -1)
    n1->Pfe(+1);
else
    n1->Pfe(0);
n2->Pfe(0);
return n2;
}

```

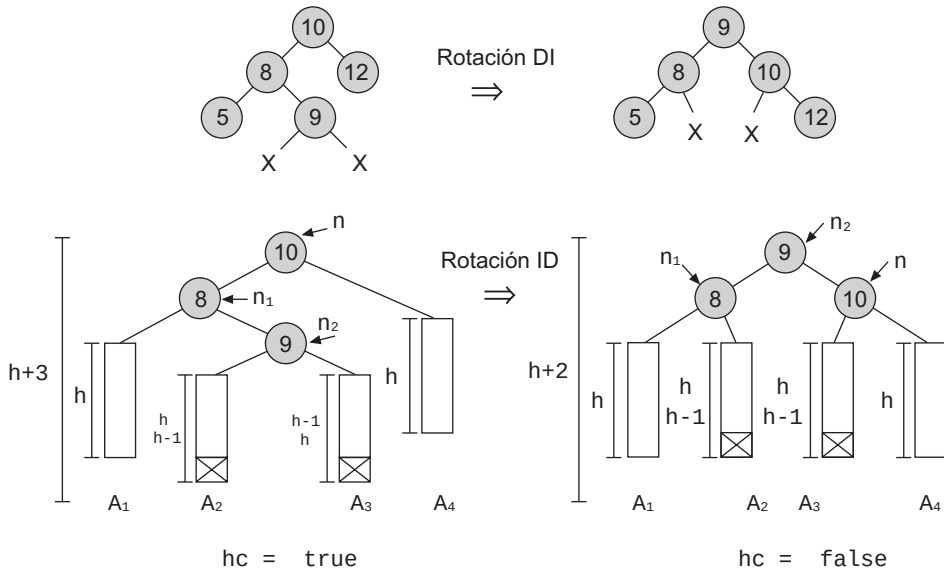


Figura 17.18. Rotación ID. Esquema gráfico. Árbol deja de aumentar en altura⁵.

```

NodoAvl* ArbolAvl::rotacionID(NodoAvl* n, NodoAvl* n1)
{
    //Figura 17.18
    NodoAvl* n2;
    n2 = n1->subarbolDcho();
    n->ramaIzdo(n2->subarbolDcho());
    n2->ramaDcho(n);
    n1->ramaDcho(n2->subarbolIzdo());
    n2->ramaIzdo(n1);
    // actualización de los factores de equilibrio
    if (n2->Ofe() == +1)
        n1->Pfe(-1);
}

```

⁵ Ibid, [KRUSE 94].

```

else
    n1->Pfe(0);
if (n2->Ofe() == -1)
    n->Pfe(1);
else
    n->Pfe(0);
n2->Pfe(0);
return n2;
}

```

Inserción con balanceo

El método público `insertarAvl()` es el interfaz de la operación, llamada a la función interna privada, recursiva, que realiza la operación y devuelve la raíz del nuevo árbol.

```

NodoAvl* ArbolAvl::insertarAvl(NodoAvl* raiz, Tipoelemento dt, bool &hc)
{
    NodoAvl *n1;

    if (raiz == NULL)
    {
        raiz = new NodoAvl(dt);
        hc = true;
    }
    else if (dt < raiz->valorNodo())
    {
        NodoAvl *iz;
        iz = insertarAvl(raiz->subarbolIzdo(), dt, hc);
        raiz->ramaIzdo(iz);
        // regreso por los nodos del camino de búsqueda
        if (hc) // siempre se comprueba si creció en altura
        {
            // decrementa el fe por aumentar la altura de rama izquierda
            switch (raiz->Ofe())
            {
                case 1: // tenía +1 y creció su izquierda
                    raiz->Pfe(0);
                    hc = false; // árbol deja de crecer
                    break;
                case 0: // tenía 0 y creció su izquierda
                    raiz->Pfe(-1); // árbol sigue creciendo
                    break;
                case -1: // aplicar rotación a la izquierda
                    n1 = raiz->subarbolIzdo();
                    if (n1->Ofe() == -1)
                        raiz = rotacionII(raiz, n1);
                    else
                        raiz = rotacionID(raiz, n1);
                    hc = false; // árbol deja de crecer en ambas rotaciones
            }
        }
    }
    else if (dt > raiz->valorNodo())
    {
        NodoAvl *dr;

```

```

dr = insertarAvl(raiz->subarbolDcho(), dt, hc);
raiz->ramaDcho(dr);
// regreso por los nodos del camino de búsqueda
if (hc) // siempre se comprueba si creció en altura
{
    // incrementa el fe por aumentar la altura de rama izquierda
    switch (raiz->Ofe())
    {
        case 1: // aplicar rotación a la derecha
            n1 = raiz->subarbolDcho();
            if (n1->Ofe() == +1)
                raiz = rotacionDD(raiz, n1);
            else
                raiz = rotacionDI(raiz, n1);
            hc = false; // árbol deja de crecer en ambas rotaciones
            break;
        case 0: // tenía 0 y creció su derecha
            raiz->Pfe(+1); // árbol sigue creciendo
            break;
        case -1: // tenía -1 y creció su derecha
            raiz->Pfe(0);
            hc = false; // árbol deja de crecer
    }
}
else
    throw "No puede haber claves repetidas ";
return raiz;
}

```

EJERCICIO 17.2. Se quiere formar un árbol binario de búsqueda equilibrado de altura 5. El campo dato de cada nodo que sea una referencia a un objeto que guarda un número entero, que será la clave de búsqueda. Una vez formado el árbol, mostrar las claves en orden creciente y el número de nodos de que consta el árbol.

La formación del árbol equilibrado se puede hacer con repetidas llamadas a la función miembro `insertarAvl()`, de la clase `ArbolAvl` que realiza las llamadas, cuando es necesario, a las *rotaciones* implementadas como métodos privados de la clase. La condición para terminar la formación del árbol está expuesta en el enunciado: la altura del árbol sea igual a 5. Por ello, se escribe la función miembro pública `altura()` de la clase `ArbolAvl` para determinar dicho parámetro. También se escribe la función miembro pública `visualizar()`, que es un recorrido en inorden, para mostrar las claves en orden creciente y a la vez contar los nodos visitados. Se escribe, además, la función miembro que encuentra el número de nodos de un árbol, así como un programa principal que realiza las correspondientes llamadas

```

int mayor (int x, int y)
{ // calcula el mayor de los números que recibe como parámetro
    return (x > y ? x : y);
}

int ArbolAvl::altura(NodoAvl* r)
{ // método público que decide la altura de un árbol apuntado por r
    if (r != NULL)
        return mayor(altura(r->subarbolIzdo()),

```

```

        altura(r->subarbolDcho())) + 1;
    else
        return false;
}
int ArbolAvl::cuantos()
{ // método público que calcula el número de nodos de un árbol
    return cuantos(raiz);
}
int ArbolAvl::cuantos (NodoAvl* r)
{ // método privado que calcula el número de nodos apuntado por r.
    if (r)
    {
        int cuantosIzquierda, cuantosDerecha;
        cuantosIzquierda = cuantos(r -> subarbolIzdo());
        cuantosDerecha = cuantos(r -> subarbolDcho());
        return cuantosIzquierda + cuantosDerecha + 1;
    }
    else
        return 0;
}
void ArbolAvl::dibujarArbol(NodoAvl *r, int h)
{
    // escribe las claves del árbol estableciendo separación entre nodos
    int i;
    if (r != NULL)
    {
        dibujarArbol(r->subarbolIzdo(), h + 1);
        for (i = 1; i <= h; i++)
            cout << " ";
        cout << r->valorNodo() << endl;
        dibujarArbol(r->subarbolDcho(), h + 1);
    }
}

int main
{
    ArbolAvl a; const int TOPE = 999;
    int numNodos;
    randomize();
    while (a.altura(a.Oraiz()) < 5)
    {
        a.insertarAvl(random(TOPE)+1);
    }
    numNodos = a.cuantos();
    cout << "\n Número de nodos: " << numNodos << endl;
    a.dibujarArbol(a.Oraiz(), 1);
    return 0;
}

```

17.5. DEFINICIÓN DE UN ÁRBOL B

Cuando se tiene un conjunto de datos masivo, por ejemplo el conjunto 1.000.000 de clientes de un banco, los registros no pueden estar en memoria principal, y se ubican en memoria auxiliar, normalmente en disco. Los accesos a disco son *críticos*, consumen recursos y necesitan

notablemente más tiempo que las instrucciones en memoria, se necesita reducir al mínimo el número de accesos a disco. Para conseguir esto se emplean árboles de búsqueda *m*-arios, que ya no tienen dos ramas como los binarios, sino que pueden tener hasta *m* ramas o subárboles descendientes, además las claves se organizan a la manera de los árboles de búsqueda; el objetivo es que la altura del árbol sea lo suficientemente pequeña ya que el número de iteraciones, y, por tanto, de acceso a disco, de la operación de búsqueda depende directamente de la altura. Un tipo particular de estos árboles son los árboles *B*, también los denominados *B+* y *B** que proceden de pequeñas modificaciones del anterior.

Los árboles *B* son árboles *m*-arios, cada nodo tiene como máximo *m* ramas, no tienen subárboles vacíos y siempre están perfectamente equilibrados. Cada nodo (en árboles *B* se acostumbra a denominar *página*) es una unidad a la que se accede en bloque. Las estructura de datos que representa un árbol *B* de orden *m* tiene las siguientes características:

- Todas las páginas hoja están en el mismo nivel.
- Todos las páginas internas, menos la raíz, tienen a lo sumo *m* ramas (no vacías) y como mínimo $m/2$ ramas.
- El número de claves en cada página interna es uno menos que el número de sus ramas, y estas claves dividen las de las ramas a manera de un árbol de búsqueda.
- La raíz tiene como máximo *m* ramas, puede llegar a tener hasta 2 y ninguna si el árbol consta de la raíz solamente.

Los árboles *B* más utilizados son los de orden 5, un orden mayor aumenta considerablemente la complejidad de los algoritmos de inserción y de borrado, un orden menor disminuye la eficacia de la localización de claves. Según la definición dada, el máximo número de claves de una página o nodo es 4 y el máximo de ramas o subárboles es 5.

La Figura 17.19 muestra un árbol *B* de orden 5, las claves de búsqueda son valores enteros. El árbol tiene 3 niveles, todas las páginas contienen 2, 3 o 4 claves. La raíz es la excepción, sólo tiene 1 clave. Todas las páginas, que son *hoja* del árbol, están en el nivel más bajo de éste, en la figura el nivel 3. Las claves mantienen una ordenación de izquierda a derecha dentro de cada página. Estas claves dividen a los nodos descendientes a la manera de un árbol de búsqueda, claves de nodo izquierdo menores, claves de nodo derecho mayores. Esta organización supone una extensión natural de los árboles binarios de búsqueda. La forma de localizar una clave en el árbol *B* consiste en seguir un camino de búsqueda, que se determina de igual manera que en los árboles binarios, con una adaptación a la característica de que cada nodo tiene $m-1$ claves.

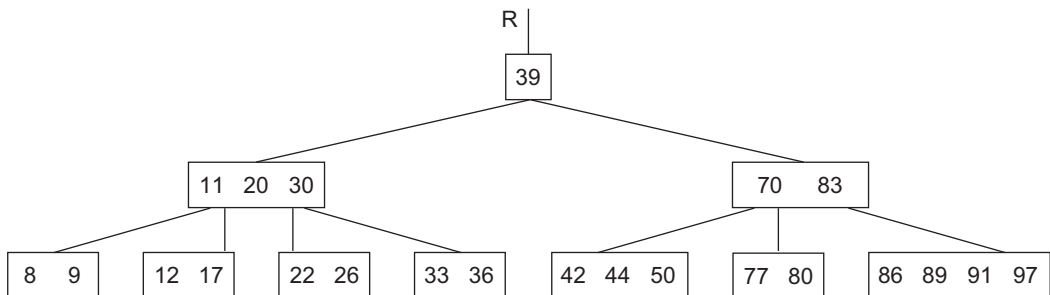


Figura 17.19. Árbol B de orden 5 de claves enteras.

17.6. TAD ÁRBOL B Y REPRESENTACIÓN

A la forma de organizar claves en la estructura árbol *B*, se le asocian una serie de operaciones básicas para poder buscar, añadir y eliminar claves. En definitiva, un árbol *B* es un tipo abstracto de datos con las siguientes operaciones básicas:

Crear	Inicializa el árbol <i>B</i> como árbol vacío.
Buscar	Dada una clave realiza la búsqueda de la clave. Devuelve la dirección del nodo y la posición en éste.
Insertar	Añade una nueva clave al árbol <i>B</i> . El árbol resultante sigue teniendo las características de árbol <i>B</i> .
Eliminar	Borra una clave del árbol <i>B</i> . El árbol resultante mantiene las características de árbol <i>B</i> .

17.6.1. Representación de una página

Los elementos de un árbol *B* son los nodos o *páginas* del árbol. Una *página* guarda las claves y las direcciones de las ramas de *páginas descendientes*. De manera natural surge la utilización de dos arrays dinámicos, y un atributo adicional cuenta que en todo momento contenga el número de claves de la *página*. El atributo privado *m* contiene siempre el número de páginas descendientes máximo que puede contener la página. La clase *Pagina* declara los dos arrays dinámicos de claves y punteros a páginas con visibilidad *protegida*, para que las operaciones tengan restricciones de acceso. El constructor crea los arrays, ajustados al orden del árbol que recibe como parámetro. Además, el método *nodoLLeno()*, devuelve *verdadero* si el número de claves es *m*-1 (máximo de claves de una página). El método *nodoSemiVacio()*, devuelve *verdadero* si el número de claves es menor que *m*/2 (mínimo de claves que puede haber en una página). Se incluyen, además, los métodos para obtener y poner las claves y ramas de cada uno de los atributos de la página. Por su parte las funciones miembro *Ocuenta()* y *Pcuenta()*, se encargan de permitir el acceso al atributo *cuenta* y la modificación del propio atributo.

```
typedef int tipoClave;
class Pagina;
typedef Pagina * PPagina;
class Pagina
{
protected:
    tipoClave *claves; // puntero array de claves variables
    PPagina *ramas;    // puntero array de punteros a páginas variable
    int cuenta;        // número de claves que almacena la página
private:
    int m;             // máximo número de claves que puede almacenar la página
public:
    // crea una página vacía de un cierto orden dado
    Pagina (int orden)
    {
        m = orden;
        claves = new tipoClave[orden];
        ramas = new PPagina[orden];
        for (int k = 0; k <= orden; k++)
            ramas[k] = NULL;
    }
}
```

```

// decide si un nodo está lleno
bool nodoLleno()
{
    return (cuenta == m - 1);
}
// decide si una página tiene menos de la mitad de claves
bool nodoSemiVacio()
{
    return (cuenta < m / 2);
}
// obtener la clave que ocupa la posición i en el array de claves
tipoClave Oclave(int i){ return claves[i];}
// cambiar la clave que ocupa la posición i en el array de claves
void Pclave(int i, tipoClave clave){ claves[i] = clave;}
// obtener la rama que ocupa la posición i en el array de ramas
Pagina* Orama(int i){ return ramas[i];}
// cambiar la rama que ocupa la posición i en el array de ramas
void Prama(int i, Pagina * p) { ramas[i] = p;}
// obtener el valor de cuenta
int Ocuenta(){ return cuenta;}
// cambiar el valor de cuenta
void Pcuenta( int valor) { cuenta = valor;}
};

```

Nota de programación

En un *árbol B* de orden m , el número de ramas de una página *no hoja* es uno más que el de sus claves. Por ello, las ramas se indexan de 0 a $m-1$, y las claves de 1 a $m-1$. El subárbol de `ramas[0]` se corresponde con las claves descendientes menores que `claves[1]`.

17.6.2. Tipo abstracto árbol B, clase *ÁrbolB*

El interfaz de la clase *ÁrbolB* se corresponde con las operaciones fundamentales del tipo abstracto: *buscar*, *insertar*, *eliminar*. Los métodos internos de la clase son los encargados del desarrollo de las operaciones del *tipo abstracto*. La clase tiene el atributo *orden* y la referencia al raíz del árbol como un puntero a la clase *Página*. Los constructores de la clase establece el orden del árbol e inicializa la raíz a NULL (*árbol vacío*). Se incluyen, además, los métodos públicos encargados de acceder y modificar el orden del árbol y la raíz del árbol.

```

class ArbolB
{
protected:
    int orden;
    Pagina *raiz;
public:
    ArbolB()
    {
        orden = 0;
        raiz = NULL;
    };
    ArbolB(int m)

```

```

{
    orden = m;
    raiz = NULL;
}
bool arbolBvacio()
{
    return raiz == NULL;
}
Pagina * Oraiz(){ return raiz;}
void Praiz( Pagina * r){ raiz = r;}
int Oorden(){ return orden;}
void Porden(int ord){ orden = ord;}
void crear() { orden = 0; raiz = NULL;}
Pagina* buscar(tipoClave cl, int &n);
void insertar(tipoClave cl);
void eliminar(tipoClave cl);
};

```

17.7. FORMACIÓN DE UN ÁRBOL B

Las claves que se añaden a un árbol B siempre se insertan a partir de un nodo hoja, como ocurre en los árboles binarios. Además, al estar un árbol B perfectamente equilibrado, todas las hojas de un árbol B se encuentren en el mismo nivel, esto impone el comportamiento característico de los árboles B: *crecen “hacia arriba”, crecen en la raíz*. Los pasos a seguir para añadir una nueva clave en un árbol B:

- Búsqueda de la clave a insertar en el árbol. Se sigue el *camino de búsqueda* que determina las claves de los nodos.
- En el caso de que la clave no esté en el árbol, la búsqueda termina en un nodo *hoja*. Entonces, empieza el proceso de inserción de la clave.
- De no estar lleno el nodo hoja, la inserción es posible en ese nodo y termina la inserción.
- Si está llena la *hoja*, la inserción en ella no es posible. Ahora se pone de manifiesto el comportamiento característico de los árboles B, se divide el nodo en dos en el mismo nivel del árbol, excepto la clave *mediana* que no se incluye en ninguno de los dos nodos, sino que *sube* en el árbol por el *camino de búsqueda* para, a su vez, repetir el proceso de inserción en el nodo *antecedente*. Por esto **un árbol B crece hacia arriba**; esta *ascensión* de la clave *mediana* puede propagarse y llegar al nodo raíz, entonces éste se divide en dos nodos y la clave enviada hacia arriba se convierte en una nueva raíz. Ésta es la forma que tiene el *árbol B* de crecer en altura.

17.7.1. Creación de un árbol B de orden 5

A continuación se va a seguir los pasos de formación de un *árbol B* de orden 5, para facilitar la comprensión las claves son números enteros. Al ser de orden 5 el número máximo de claves de un nodo es 4 y el máximo de ramas 5.

Supóngase que las claves que se insertan:

6 11 5 4 8 9 12 21 14 10 19 28 3 17 32 15 16 26 27

- Con las cuatro primeras se completa el primer nodo, en orden creciente a la manera de árbol de búsqueda. La Figura 17.20(a) muestra este nodo.

- La clave siguiente, 8, encuentra el nodo ya lleno. La clave *mediana* de las cinco claves es 6. El nodo lleno se divide en dos, la clave mediana “*sube*” y como no hay nodo antecedente se crea otro nodo con la clave *mediana*, es la nueva raíz del árbol. El árbol ha crecido en altura como puede observarse en la Figura 17.20(b).

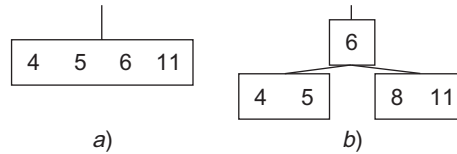


Figura 17.20. Formación de un árbol B de orden 5.

- Las siguientes claves 9, 12 se insertan en el nodo derecha de la raíz, según el criterio de búsqueda, ambas claves son mayores que 6. La Figura 17.21a) muestra el árbol después de esta inserción.
- La clave 21 *baja* por el *camino de búsqueda*, rama derecha del nodo raíz. El nodo hoja que le corresponde está lleno; por consiguiente, el nodo se parte en dos y la clave mediana, 11, asciende por el *camino de búsqueda* para ser insertada en el nodo antecedente, en este caso la raíz. El proceso se repite, ahora si hay *hueco* para que la clave 11 sea insertada. La Figura 17.21b) muestra el árbol resultante después de la creación del nuevo nodo.

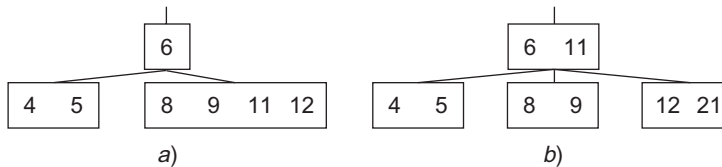


Figura 17.21. Inserción de nuevas claves en un árbol B de orden 5.

- Las siguientes claves 14, 10, 19 se insertan en los nodos *hoja* que les corresponde, según el *camino de búsqueda*.
- Al insertar la siguiente clave, 28, el *camino de búsqueda* determina que se inserte en un nodo que está lleno (nodo derecho del último nivel). De nuevo se produce el proceso de división del nodo y ascensión, por el *camino de búsqueda*, de la clave *mediana* que ahora es 19. El antecedente, nodo raíz, tiene dos claves y, por consiguiente, se inserta 19.
- Las claves que vienen a continuación, 3, 17, 32, 15 son insertadas en los nodos *hoja* que determina el *camino de búsqueda*. Cada uno de estos nodos tiene un número de claves menor que el máximo, $m-1$, por ello cada clave se inserta directamente en la hoja, en la posición que le corresponde según su valor. La Figura 17.22 representa al árbol después de estas inserciones.

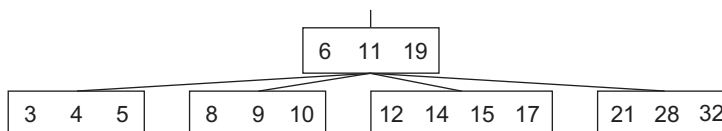


Figura 17.22. Representación de árbol B de orden 5.

- Ahora hay que añadir la clave 16; “*baja*” por el *camino de búsqueda* (rama derecha de clave 11). El nodo donde va a ser insertado está lleno, se produce la división del nodo y la clave mediana, 15, “*sube*” por el *camino de búsqueda* para que sea añadida en el nodo antecedente. Como éste no está lleno, se inserta la clave 15; el nodo raíz ha quedado completo (Figura 17.23).

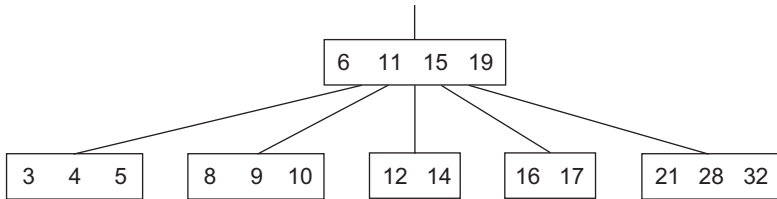


Figura 17.23. Árbol B de orden 5 después de una división de nodo .

- Al insertar la clave 26 se completa el nodo más a la derecha del último nivel. Por último, la clave 27 “*baja*” por el mismo *camino de búsqueda*, el nodo *hoja* que le corresponde está completo, entonces se divide en dos y “*sube*” la clave *mediana*, 27, al nodo *padre*. Ocurre que este también está completo, se repite el proceso de división en dos nodos, y “*sube*” la nueva clave *mediana*, 15. Como el nodo dividido es el raíz, con esta clave *mediana* se forma un nuevo nodo raíz: el árbol ha crecido en altura. El *árbol crece hacia arriba*, hacia la raíz.

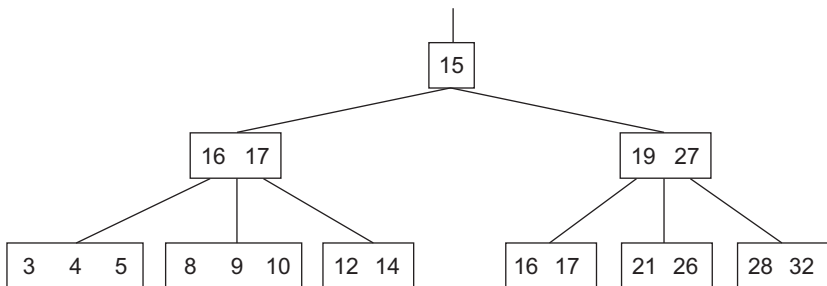


Figura 17.24. Árbol B de orden 5 con todas las cla ves.

A tener en cuenta

En el proceso de formación de un *árbol B* hay que destacar dos hechos relevantes. El primero, la división de un nodo prepara a la estructura para nuevas inserciones, ya que deja *huecos* en los nodos implicados . En segundo lugar , la cla ve que “*sube*” al nodo antecedente es la clave *mediana* del nodo lleno, no tiene por qué coincidir con la clave que se está inser tando; se puede afir mar que no impor ta el orden en que lleguen las claves en el balanceo del árbol.

17.8. BÚSQUEDA DE UNA CLAVE EN UN ÁRBOL B

Los elementos de un nodo en un árbol B son *claves de búsqueda*, dividen a las ramas inmediatas, rama izquierda y derecha, a la manera de un árbol de búsqueda. Por consiguiente, el algoritmo es similar al de búsqueda en un árbol binario ordenado, salvo que en los árboles B cuando se está analizando una *página* hay que inspeccionar las claves de que consta. La inspección da como resultado la posición de la clave, o bien la rama por donde seguir buscando (*camino de búsqueda*).

El método que implementa la operación, si encuentra la clave, devuelve la dirección del nodo que contiene a la clave y la posición que ocupa. Si la clave no está en el árbol, devuelve NULL. La búsqueda en cada nodo la realiza el método auxiliar `buscarNodo()`.

17.8.1. `buscarNodo()`

El método devuelve true si encuentra la clave en el árbol. Además, en el argumento *k* se obtiene la posición que ocupa la clave en la *página*, o bien la rama por donde continuar el proceso de búsqueda. Aprovecha la ordenación de las claves en el nodo, de tal forma que inspecciona las claves de la *página* en orden descendente y termina cuando `claves[index] <= cl`, (`actual->Oclave(index) <= cl`) así se asegura que *index* es la posición que ocupa la clave o el índice de la rama del árbol en la que puede ser encontrada.

```
bool ArbolB::buscarNodo(Pagina* actual, tipoClave cl, int & k)
{
    int index;
    bool encontrado;
    if (cl < actual->Oclave(1))
    {
        encontrado = false;
        index = 0;
    }
    else
    {
        // orden descendente
        index = actual->Ocuanta();
        while (cl < actual->Oclave(index) && (index > 1))
            index--;
        encontrado = cl == actual->Oclave(index);
    }
    k = index;
    return encontrado;
}
```

17.8.2. `buscar()`

Este método privado de la clase `ArbolB` controla el proceso de búsqueda. El método *baja* por las ramas del árbol hasta encontrar la clave. El método se implementa recursivamente, la condición para terminar de hacer llamadas recursivas es que se haya localizado la clave, o bien el puntero a la página *actual* es a NULL, por tanto, no hay más nodos en los que buscar. Devuelve la referencia al nodo donde se encuentra la clave, o bien NULL. La función miembro está sobrecargada; la función interna privada realiza la búsqueda desde la raíz.

```

Pagina* ArbolB::buscar( tipoClave cl, int &n)
{
    return buscar( raiz, cl, n);
}

Pagina* ArbolB::buscar(Pagina* actual, tipoClave cl, int &n)
{
    if (actual == NULL)
    {
        return NULL;
    }
    else
    {
        bool esta = buscarNodo(actual, cl, n);
        if (esta) // la clave se encuentra en el nodo actual
            return actual;
        else
            return buscar(actual->Orama(n), cl, n); //llamada recursiva
    }
}

```

17.9. INSERCIÓN EN UN ÁRBOL B

Debido a que las claves de un árbol B están organizadas como un árbol *m-ario de búsqueda*, la operación que inserta una clave en un árbol B sigue la misma estrategia que la inserción en un árbol binario de búsqueda. La nueva clave siempre se inserta en una hoja, para lo cual *baja por el camino de búsqueda*, hasta alcanzar el nodo hoja. Esta primera parte del algoritmo utiliza el método `buscarNodo()` para determinar la rama por donde *bajar*.

Una vez en el nodo hoja, si no está lleno, se inserta directamente en la posición que devuelve `buscarNodo()`, de tal forma que la clave queda ordenada. El método `meterPagina()` realiza esta parte de la operación.

Sólo si el nodo está lleno la inserción afecta a la estructura del árbol, ya que se ha de crear un nuevo nodo y, además, para que el árbol mantenga las características de *árbol B*, *asciende la clave mediana al nodo antecedente*. El método `dividirNodo()` es donde se implementan las acciones de esta parte del algoritmo.

La formulación recursiva es la más adecuada para reflejar el proceso de propagación, *hacia arriba*, en la *división de los nodos*, debido a que al retornar la llamada recursiva se *regresa por el camino de búsqueda*, se pasa por los nodos en sentido inverso a como se *bajó*.

El método interno `insertar()` tiene dos argumentos, la clave y el nodo raíz; ésta puede cambiar, si el árbol crece en altura y por ello devuelve la referencia al raíz. Pasa control a `empujar()` que, con una estrategia recursiva, realiza el proceso de inserción. Los métodos `meterPagina()` y `dividirNodo()` son llamados desde `empujar()`, según que la inserción se realice en un nodo con posiciones vacías o lleno, respectivamente.

17.9.1. Método `insertar()`

Se encarga de crear una nueva raíz si la *propagación, hacia arriba*, del proceso de división llega al actual raíz (el árbol aumenta su altura). El método está sobrecargado. En el primer caso,

el método es público y realiza la llamada al método `insertar()` pero teniendo como parámetro la raíz del árbol. Véase Figura 17.25.

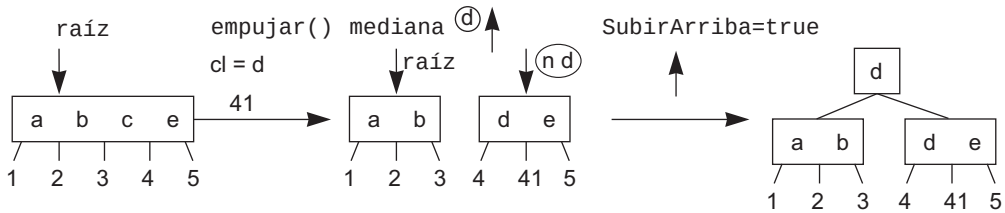


Figura 17.25. Crecimiento en altura de un árbol.

```
// método público
void ArbolB::insertar(tipoClave cl)
{
    raiz = insertar(raiz, cl);
}
//método privado
Pagina* ArbolB::insertar(Pagina* raiz, tipoClave cl)
{
    bool subeArriba;
    int mediana;
    Pagina* nd;
    subeArriba = empujar(raiz, cl, mediana, nd);
    if (subeArriba)
    {
        // El árbol crece en altura por la raíz.
        // sube una nueva clave mediana y un nuevo hijo derecho nd
        // en la implementación se mantiene que las claves que son
        // menores que mediana se encuentran en raiz y las mayores en nd
        Pagina* p;
        p = new Pagina(orden); // nuevo nodo
        p->Pcuenta(1);          // tiene una sola clave
        p->Pclave(1, mediana);
        p->Prama(0, raiz);      // claves menores
        p->Prama(1, nd);        // claves mayores
        raiz = p;
    }
    return raiz;
}
```

17.9.2. Método `empujar()`

Es el método más importante, controla la realización de las tareas de búsqueda y posterior inserción. Lo primero que hace es “bajar” por el *camino de búsqueda* hasta llegar a una rama vacía. A continuación, se prepara para “subir” (activa el indicador `subeArriba`) por las páginas del camino y realizar la inserción. En esta *vuelta atrás*, primero se encuentra con la *Página* hoja, si hay *hueco* mete la clave y el proceso termina. De estar completa, llama a divi-

`dirNodo()` que crea una nueva *Página* para repartir las claves; la clave *mediana* sigue “*subiendo*” por el *camino de búsqueda*.

La simulación de “*bajar*” y luego “*subir*” por el *camino de búsqueda* se implementa fácilmente mediante las llamadas recursivas de `empujar()`, de tal forma que cuando las llamadas retornan, se está volviendo (“*subiendo*”) a las de la *Páginas* por los que pasó anteriormente: *regresa por el camino de búsqueda*.

La inserción de la clave mediana en la *Página* antecesor, debido a la división de la *Página*, puede, a su vez, causar el desbordamiento de la misma, dando como resultado la propagación del proceso de partición *hacia arriba*, pudiendo llegar a la raíz. Ésta es la única forma de que aumente la altura del árbol B. En el caso de que la altura del árbol aumente es cuando el método `insertar()` crea un nuevo nodo que almacena una sola clave y dos punteros que apuntan a *Páginas* con claves menores y mayores respectivamente.

El método `empujar()` es privado y retorna el valor verdadero si hay clave que sube hacia arriba en cuyo caso se retorna en mediana, y además las claves más pequeñas están apuntadas por el puntero a la *página* actual y las mayores por el puntero a *página* nuevo.

```
bool ArbolB::empujar(Pagina* actual, tipoClave cl,
                    tipoClave &mediana, Pagina *& nuevo)
{
    int k ;
    bool subeArriba = false;
    if (actual == NULL)
    { // envía hacia arriba la clave cl y su rama derecha NULL
      // para que se inserte en la Página padre
      subeArriba = true;
      mediana = cl;
      nuevo = NULL;
      // el dato Página de nuevo está a NULL
    }
    else
    {
        bool esta;
        esta = buscarNodo(actual, cl, k);
        if (esta)
            throw "\nClave duplicada";
        // siempre se ejecuta
        subeArriba = empujar(actual->Orama(k), cl, mediana, nuevo);
        // devuelve control; vuelve por el camino de búsqueda
        if (subeArriba)
        {
            if (actual->nodoLLeno()) // hay que dividir la página
                dividirNodo(actual, mediana, nuevo, k);
        }
        else
        { //cabe en la página, se inserta la mediana y su rama derecha
          subeArriba = false;
          meterPagina(actual, mediana, nuevo, k);
        }
    }
    return subeArriba;
}
```

Nota de programación

En un árbol B los elementos son claves de búsqueda, por ello no se contempla que haya elementos repetidos. Ésa es la razón para que el método *empujar* levante una excepción cuando el método *buscarNodo*() encuentra la clave que se quiere insertar.

18.5.3. Método *meterPagina*()

Este método privado de la clase *ArbolB* inserta una clave en una *Página* que tiene un número de claves menor que el máximo. El método es invocado una vez que *empujar*() ha comprobado que hay *hueco* para añadir a la *Página* una nueva clave. Se le pasa, como argumentos, la dirección de la *Página*, la clave, la dirección de la rama con la *Página* sucesor, y la posición a partir de la cual se inserta. Véase Figura 17.26

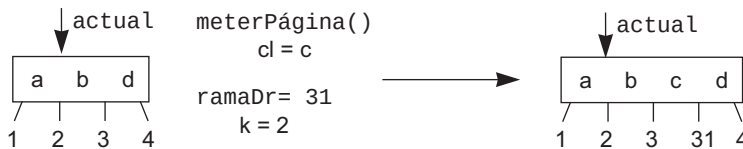


Figura 17.26. Proceso de Meter en una página.

```
void ArbolB::meterPagina(Pagina* actual, tipoClave cl,
                        Pagina *ramaDr, int k)
{
    // desplaza a la derecha los elementos para hacer un hueco
    for (int i = actual->Ocuenta(); i >= k + 1; i--)
    {
        actual->Pclave(i + 1, actual->Oclave(i));
        actual->Prama(i + 1, actual->Orama(i));
    }
    // pone la clave y la rama derecha en la posición k+1
    actual->Pclave(k + 1, cl);
    actual->Prama(k + 1, ramaDr);
    // incrementa el contador de claves almacenadas
    actual->Pcuenta(actual->Ocuenta()+1) ;
}
```

17.9.4. Método *dividirNodo*()

Este método resuelve el problema de que la *Página* donde se debe insertar la clave esté llena. Virtualmente, la *Página* se divide en dos y la clave mediana es enviada *hacia arriba*, para una *re-inserción* posterior en una *Página* padre o bien en una nueva raíz en el caso de que el árbol deba crecer en altura. Para ello, se crea una nueva *Página* a la que se desplazan las claves mayores de la mediana y sus correspondientes ramas, dejando en la *Página* original las claves menores.

El algoritmo, en primer lugar, encuentra la posición que ocupa la clave mediana; después mueve claves y ramas a la *Página* nueva y, según la posición k de inserción, mete la clave en la *Página* original o en la nueva. Por último, “*extrae*” la clave mediana que siempre se deja en el nodo original. Con la técnica de llamadas recursiva, el método `empujar()` insertará, posteriormente, la clave mediana en la *Página* antecedente. El argumento `mediana` recibe la clave que se inserta y se actualiza con la nueva mediana; de igual forma, el argumento `nuevo` recibe la rama derecha de la clave a insertar y se actualiza con la *Página* creada.

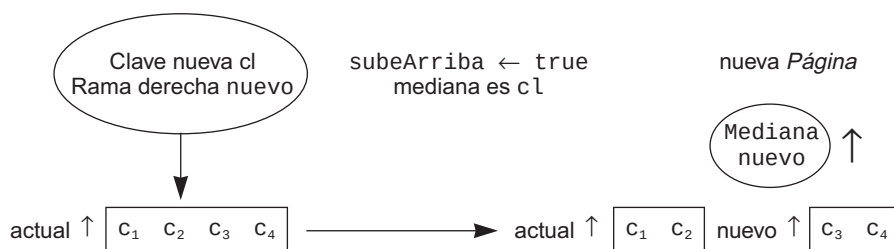


Figura 17.27. Proceso de partición de una *Página*.

```
void ArbolB::dividirNodo(Pagina* actual, tipoClave &mediana,
                          Pagina * &nuevo, int pos)
{
    int i, posMdna, k;
    Pagina *nuevaPag;
    k = pos;
    // posición de clave mediana
    posMdna = (k <= orden/2) ? orden/2 : orden/2 + 1;
    nuevaPag = new Pagina(orden);
    for (i = posMdna + 1; i < orden; i++)
    {
        /* desplazada la mitad derecha a la nueva Página, la clave
           mediana se queda en Página actual */
        nuevaPag->Pclave(i - posMdna, actual->Oclave(i));
        nuevaPag->Prama(i - posMdna, actual->Orama(i));
    }
    nuevaPag->Pcuenta ((orden - 1) - posMdna); // claves de nueva Página
    actual->Pcuenta(posMdna); // claves en Página origen
    // inserta la clave y rama en la Página que le corresponde
    if (k <= orden / 2)
        meterPagina(actual, mediana, nuevo, pos); // en Página origen
    else
    {
        pos = k - posMdna;
        meterPagina(nuevaPag, mediana, nuevo, pos); // en Página nueva
    }
    // extrae clave mediana de la Página origen
    mediana = actual->Oclave(actual->Ocuenta());
    // Rama0 del nuevo nodo es la rama de la mediana
    nuevaPag->Prama(0, actual->Orama(actual->Ocuenta()));
    actual->Pcuenta(actual->Ocuenta() - 1); // se quita la mediana
    nuevo = nuevaPag; // devuelve la nueva Página
}
```


17.9.5. Método escribir()

El método público `escribir()` realiza la tarea de visualizar en pantalla un `ArbolB` pequeño. Al igual que otros métodos está sobrecargado, el primero de ellos realiza la llamada al segundo con un argumento más que es la raíz del árbol. El parámetro `h` de `escribir` se usa para formatear la salida y que se pueda visualizar en la pantalla el árbol.

```
void ArbolB::escribir( )
{
    escribir (raiz, 1);
}
void ArbolB::escribir( Pagina * r, int h)
{
    int i;
    if (r != NULL)
    {
        escribir( r->Orama(0), h + 5);
        for (i = 1; i <= r->Ocuenta()/2; i++)
        { // llamadas recursivas a la mitad de los subárboles
            escribir(r->Orama(i), h + 5);
            cout << endl;
        }
        // visualización de las claves de la página apuntada por r
        for (i = 1; i <= r->Ocuenta(); i++)
        {
            for (int j = 0; j <= h; j++)
                cout << " ";
            cout << r->Oclave(i) << endl;
        }
        // llamadas recursivas a la otra mitad de los subárboles
        for (i = r->Ocuenta() / 2 + 1; i <= r->Ocuenta(); i++)
            escribir(r->Orama(i), h + 5);
        cout << endl;
    }
}
```

17.10. LISTADO DE LAS CLAVES DE UN ÁRBOL B

Recorrer una estructura de datos implica *visitar* cada uno de los elementos de que consta, así ocurre con los vectores, con las listas lineales y circulares, En los árboles se distinguen distintos tipos de recorrido, en profundidad y en anchura. **El recorrido en profundidad se puede hacer en preorden, inorden y postorden.** En los árboles B se puede aplicar los mismos criterios para visitar sus claves, con la particularidad de que un nodo (*Página*) de un árbol B hay, normalmente, más de una clave y más de dos subárboles.

Un **recorrido** que resulta interesante en un árbol B es el que **visita las claves del árbol en orden creciente.** Como cada clave de la *Página* del árbol B divide a las de sus ramas adyacentes a la manera de un árbol de búsqueda, se aplica el recorrido en *inorden* de un árbol binario, con una pequeña adaptación, para *visitar* en orden creciente las claves de un árbol B. La implementación se realiza con dos métodos: `listaCreciente()` que es el interfaz público de la clase `ArbolB`, y `inOrden()` de la clase `ArbolB` como método privado que realiza el recorrido por las claves.

```

void ArbolB::listarCreciente()
{
    inOrden(raiz) ;
}
void ArbolB::inOrden(Pagina *r)
{
    if (r)
    {
        inOrden(r->Orama(0));
        for (int k = 1; k <= r->Ocuenta(); k++)
        {
            cout << r->Oclave(k) << " ";
            inOrden(r->Orama(k));
        }
    }
}

```

Nota de programación

El recorrido en *inorden* del árbol B se puede implementar iterativamente. Para ello se utiliza una *Pila* en la que se guardan las referencias a las Páginas del árbol.

EJERCICIO 17.3. Escribir una aplicación para gestionar un árbol B de orden 5, utilizando las operaciones del TAD ya implementadas en la clase ArbolB. Deben realizarse estas acciones:

- *Dar de alta claves.*
- *Buscar una clave dada.*
- *Eliminar una clave dada.*
- *Listar las claves en orden creciente.*

El programa crea un ArbolB de con claves de tipo entero. Las operaciones se presentan en forma de menú; la opción “*dar de alta claves*” genera *n* claves aleatoriamente, en el rango de 1 a 9999, que se añaden al árbol llamando al método *insertar()*. Las acciones de *buscar*, *eliminar* y *listar* se corresponden con las operaciones implementadas en la clase.

Codificación. Véase página web del libro

Lecturas recomendadas para profundizar en el tema

En la lectura recomendada Anexo A puede consultar una ampliación sobre “Eliminación en árboles AVL”.

En la lectura recomendada Anexo B puede consultar una ampliación sobre “Eliminación en árboles B”.

RESUMEN

Los árboles binarios de búsqueda son estructuras de datos importantes que permiten localizar una *clave de búsqueda* con un coste logarítmico. Sin embargo, el tiempo de búsqueda puede crecer hasta hacerse lineal si el árbol está degenerado; en ese sentido, la eficiencia de las operaciones dependen de lo aleatorias que sean las claves de entrada. Para evitar este problema surgen los árboles de búsqueda equilibrados, llamados árboles AVL. La eficiencia de la *búsqueda* en los árboles equilibrados es mayor que en los árboles de búsqueda no equilibrados.

Un árbol binario equilibrado es un árbol de búsqueda caracterizado por diferir las alturas de las ramas derecha e izquierda del nodo raíz a lo sumo en uno, y que los subárboles izquierdo y derecho son, a su vez, árboles equilibrados.

Las operaciones de los árboles de búsqueda son también operaciones de los árboles equilibrados. Las operaciones de inserción y de borrado añaden o eliminan, respectivamente, un nodo en el árbol de igual forma que en los árboles de búsqueda no equilibrados. Además, es necesario incorporar una segunda parte de reestructuración del árbol para asegurar el equilibrio de cada nodo. La solución que se aplica ante un desequilibrio de un nodo viene dada por las *rotaciones*. Se pueden aplicar dos tipos de rotaciones, *rotación simple* y *rotación doble*; el que se aplique una u otra depende del nodo desequilibrado y del *equilibrio* del hijo que se encuentra en la rama más alta.

Las operaciones de inserción y de borrado en árboles binarios equilibrados son más *costosas* que en los no equilibrados, debido a que el algoritmo debe *retroceder* por el *camino de búsqueda*, para actualizar el factor de equilibrio de los nodos que lo forman y al *coste* de la rotación cuando se viola la condición de equilibrio. Por el contrario, la operación de búsqueda es, de promedio, menos costosa que en los árboles binarios no equilibrados.

Los *árboles B* tienen una serie de características que permiten que la búsqueda de una clave sea muy eficiente, por ello, *si las claves se encuentran en un disco, el número de accesos al disco disminuye notablemente*. En aplicaciones que acceden a archivos que están en disco, el mayor tiempo de proceso es el de las operaciones de *entrada/salida*. Reducir el número de pasos para buscar una clave, o para insertar, supone reducir el número de operaciones de entrada y salida y, por consiguiente, el tiempo de proceso. Entonces, un árbol-B es una estructura de datos utilizada para almacenar claves (*de búsqueda*) que residen en memoria externa, tal como un disco.

El número de claves de una *Página* (nodo) en un árbol B de orden m , que no sea la raíz, es como mínimo $m/2$ y como máximo $m-1$. Además, las claves de una *Página* divide a las claves de sus ramas a la manera de árbol de búsqueda. El número de ramas descendientes de una *Página* es una más que el número de claves, excepto las *Páginas* que son hoja.

Un árbol B siempre está perfectamente equilibrado. La inserción de una nueva clave se realiza en un hoja, si está llena *asciende* la clave *mediana* a la *Página padre*, si éste también está lleno sigue *ascendiendo*, pudiendo llegar a la raíz, si también se encuentra llena se crea una nueva *Página* con la *clave mediana*. Se dice que los árboles B crecen hacia arriba, hacia la raíz. Los árboles B más utilizados son los de orden 5.

BIBLIOGRAFÍA RECOMENDADA

Aho V.; Hopcroft, J., y Ullman, J.: *Estructuras de datos y algoritmos*. Addison Wesley, 1983.

Garrido, A., y Fernández, J.: *Abstracción y estructuras de datos en C++*. Delta, 2006.

Joyanes, L., y Zahonero, L.: *Algoritmos y estructuras de datos. Una perspectiva en C*. McGraw-Hill, 2004.

Joyanes, L.; Sánchez, L.; Zahonero, I., y Fernández, M.: *Estructuras de datos en C*. Schaum, 2005.